

News-Hash: Konstruktion eines nachvollziehbaren Nachrichtenarchivs

Ein wissenschaftlicher Projektbericht zu Datenmodell, Integritätsprüfung, Quellenverarbeitung und öffentlicher Zeitverankerung

News-Hash-Projekt · Technischer Bericht · 17. August 2026

Zusammenfassung. Die digitale Nachrichtenüberlieferung ist durch nachträgliche Redaktion, wechselnde URLs und das Verschwinden einzelner Veröffentlichungen geprägt. Der vorliegende Beitrag beschreibt News-Hash als ein lokal betriebenes Nachrichtenarchiv, das konfigurierte JSON- und XML-Quellen regelmäßig abrufen, normalisiert und in zwei append-only Speicherformaten ablegt. JSONL und SQLite führen unabhängige Hash-Ketten, sodass Änderungen am Inhalt oder an der Reihenfolge nachträglich erkennbar werden. Ein tägliches Manifest verdichtet die jeweiligen Kettenenden und kann mit OpenTimestamps öffentlich zeitverankert werden, ohne Nachrichtentexte an einen externen Dienst zu übertragen. Untersucht werden die Architektur, das Datenmodell, die Codec-Schnittstelle, die Fehlersemantik, die WebUI sowie das Vertrauens- und Bedrohungsmodell. Der Beitrag argumentiert, dass die Kombination etablierter Verfahren eine belastbare technische Herkunftsspur erzeugt, ohne den Anspruch zu erheben, journalistische Wahrheit, Quellenvollständigkeit oder politische Neutralität zu beweisen.

Schlüsselwörter: Nachrichtenarchivierung, digitale Provenienz, Hash-Kette, SHA-256, OpenTimestamps, Datenintegrität

1. Einleitung

Nachrichten sind ein zentraler Bestandteil öffentlicher Wissensbildung und zugleich ein ungewöhnlich flüchtiger Informationstyp. Redaktionen aktualisieren Texte, korrigieren Überschriften, tauschen Bilder aus oder entfernen Beiträge. Suchmaschinen und soziale Plattformen bewahren häufig nur Verweise auf eine jeweils aktuelle Fassung. Für eine spätere Analyse entsteht dadurch ein Quellenproblem: Die heute erreichbare Seite muss nicht der Version entsprechen, die eine Person gestern gelesen und auf deren Grundlage sie gehandelt hat.

Ein Archiv kann auf diese Flüchtigkeit unterschiedlich reagieren. Es kann versuchen, möglichst viele Ressourcen zu kopieren, oder es kann die konkrete lokale Repräsentation so dokumentieren, dass ihre spätere Unverändertheit geprüft werden kann. News-Hash verfolgt den zweiten Schwerpunkt. Die Anwendung liest konfigurierte Quellen regelmäßig ein, erkennt bereits bekannte Meldungen, speichert neue Datensätze dauerhaft und verknüpft jeden Datensatz mit dem Hash seines Vorgängers. Dadurch wird aus einer Folge von Abrufen eine nachvollziehbare Archivspur.

Die technische Konstruktion ist bewusst pragmatisch. Sie benötigt keine eigene Blockchain und keinen permanenten externen Konsens. Die Inhalte bleiben lokal in JSONL- und SQLite-Dateien. Erst die Hashwerte der lokalen Ketten werden in einem kleinen Manifest zusammengefasst und optional öffentlich mit OpenTimestamps verankert. Die externe Infrastruktur erhält somit keinen Nachrichtenbestand, sondern nur einen kryptografischen Fingerabdruck eines lokalen Zustands.

Der Artikel behandelt News-Hash als Systementwurf und nicht als empirische Studie über Medienqualität. Im Mittelpunkt stehen die Fragen, welche Integritätsaussagen die Architektur tatsächlich ermöglicht, welche Betriebsannahmen dafür erforderlich sind und an welchen Stellen eine Interpretation über die technische Evidenz hinausgehen würde.

2. Forschungsziel und Leitfragen

Das Forschungsziel ist die Konstruktion eines kleinen, reproduzierbar betreibbaren Archivs für laufend aktualisierte Nachrichtenquellen. „Reproduzierbar“ bedeutet dabei nicht, dass jeder externe Webinhalt unter allen Umständen bitgleich erneut erzeugt werden kann. Gemeint ist vielmehr, dass Datenmodell, Hashmaterial, Speicherschlritte, Konfiguration und Prüfregeleln so beschrieben sind, dass ein Dritter die zentrale Integritätsaussage nachvollziehen kann.

Aus diesem Ziel ergeben sich fünf Leitfragen. Erstens: Wie muss ein normalisierter Datensatz aufgebaut sein, damit seine Hashberechnung unabhängig von der Eingabereihenfolge stabil bleibt? Zweitens: Wie können ein zeilenorientiertes Dateiformat und eine relationale Datenbank parallel betrieben werden, ohne eine unklare gemeinsame Transaktionsgrenze vorzutäuschen? Drittens: Wie lassen sich quellspezifische Anforderungen, etwa vollständige Artikeltexte oder Screenshots, in einen allgemeinen Importprozess integrieren? Viertens: Wie können Fehler einzelner Quellen isoliert werden, ohne den gesamten Daemon zu stoppen? Fünftens: Wie lässt sich die Existenz eines lokalen Kettenzustands unabhängig vom Betreiber zeitlich einordnen?

Die Arbeit nutzt einen konstruktionsorientierten Ansatz. Anforderungen werden in Invarianten übersetzt und anschließend in Datenmodell, Codecs, Speicherlogik, WebUI und Tests umgesetzt. Die Untersuchung betrachtet Normalbetrieb, Wiederholungsabruf, beschädigte Eingaben, Prozessabbruch und nachträgliche Änderung eines Datensatzes. Entscheidend ist dabei nicht nur, was das System erkennt, sondern auch, welche Aussagen es nicht treffen kann.

3. Begriffliche Einordnung

News-Hash verbindet drei Ebenen, die in der Diskussion über digitale Archive häufig getrennt betrachtet werden. Die erste Ebene ist die lokale Inhaltsbewahrung. Sie beantwortet die Frage, welche konkreten Felder, Bilder und Zusatzinformationen gespeichert wurden. Die zweite Ebene ist die interne Nachvollziehbarkeit. Hash-Ketten zeigen, ob eine gespeicherte Folge seit ihrer Erzeugung verändert wurde. Die dritte Ebene ist der externe Existenznachweis. Ein OpenTimestamps-Proof kann bestätigen, dass ein bestimmter Hash spätestens zu einem Zeitpunkt existierte.

Diese Ebenen dürfen nicht miteinander verwechselt werden. Ein Webarchiv kann umfangreiche Ressourcen speichern und dennoch keine nachträgliche Änderung sichtbar machen. Eine Hash-Kette kann Integrität prüfen, ohne die Quelle vollständig zu kopieren. Ein Zeitstempel kann die Existenz eines Hashes belegen, ohne den zugehörigen Nachrichtentext zu veröffentlichen oder dessen Richtigkeit zu bewerten. Die Stärke des Projekts liegt in der bewussten Kombination, nicht in einer einzelnen Komponente.

Ebenso ist die interne Kette keine Blockchain im engeren Sinn. Eine Blockchain setzt typischerweise verteilte Replikation, Konsens und ein Modell gegen konkurrierende Zustände voraus. News-Hash arbeitet zunächst mit einem lokalen Betreiber und einer lokalen Datenhaltung. Die öffentliche Zeitverankerung wird nachgelagert eingesetzt. Diese Entscheidung reduziert Komplexität, macht aber die Vertrauensannahme sichtbar: Der Betreiber ist für Abruf, Konfiguration und Veröffentlichung der Daten verantwortlich.

4. Gesamtarchitektur

Die Anwendung besteht aus einem CLI-Programm, einer Import- und Normalisierungsschicht, einer Codec-Schnittstelle, zwei Speicherschichten, einem optionalen Anchoring-Prozess und einer eingebetteten HTTP-WebUI. Die Quellen werden in `data/settings.toml` beschrieben. Jeder Quellenblock enthält einen Anzeigenamen, eine Feed-URL, einen Speichernamen, einen Codecnamen und ein individuelles Polling-Intervall. Die Verarbeitung läuft quellenbezogen, damit ein Fehler bei einem Anbieter andere Anbieter nicht blockiert.

Der Daemon übernimmt die zeitgesteuerte Ausführung. Ein einmaliger Lauf ruft jede konfigurierte Quelle einmal ab; im Daemon-Modus wird jede Quelle nach ihrem eigenen Intervall erneut verarbeitet. Die Importlogik führt Abruf, Parsing, Deduplizierung, Normalisierung und Speicherung aus. Laufzeitfehler werden pro Quelle gezählt, nach `stderr` geschrieben und im Prozessspeicher für Dashboard und Prometheus-Metriken vorgehalten.

Die WebUI ist kein separates Frontend-Deployment, sondern Teil des Prozesses. Sie greift auf SQLite zu, zeigt Quellenkarten, Kennzahlen, Meldungslisten und Detailansichten und stellt Metriken unter `/metrics` bereit. Diese enge Kopplung vereinfacht die Installation eines lokalen Archivs. Zugleich ist sie eine Sicherheitsgrenze: Die WebUI hat keine Benutzerverwaltung und keine Authentifizierung und darf daher nicht ungeschützt im öffentlichen Netz betrieben werden.

Die Architektur trennt bewusst fachliche und technische Verantwortlichkeiten. Die Normalisierung entscheidet, welche Werte ein Datensatz besitzt. Der Hash-Codec entscheidet, wie diese Werte kanonisch verarbeitet werden. JSONL und SQLite entscheiden, wie sie dauerhaft abgelegt werden. Anchoring und GitHub-Synchronisierung arbeiten erst auf den Kettenendpunkten. Diese Schichtung erlaubt eine unabhängige Prüfung und begrenzt Seiteneffekte bei Erweiterungen.

5. Quellenaufnahme und Normalisierung

Ein Quellenlauf beginnt mit dem Abruf eines JSON- oder XML-Feeds über HTTP. Das externe Format wird in eine interne Feedrepräsentation überführt. Dabei werden Titel, URL, Inhalt, Autor, Veröffentlichungszeit und Bildinformationen aus unterschiedlichen Eingabefeldern zusammengeführt. Zeitangaben werden intern als UTC-Werte behandelt und erst in der Browserdarstellung lokal interpretiert. Die Normalisierung ist nicht nur eine Komfortfunktion; sie definiert das fachliche Material, das später gehasht wird.

Vor teuren Verarbeitungsschritten prüft die Anwendung bereits bekannte `source_id`-Werte. Diese Deduplizierung reduziert Netzwerkzugriffe, Bilddownloads und unnötige Kettenelemente. Die

Entscheidung setzt allerdings voraus, dass die Kennung einer Quelle stabil und semantisch sinnvoll ist. Eine Quelle, die dieselbe Kennung für mehrere Fassungen verwendet, kann durch die Deduplizierungsregel relevante Änderungen verbergen; eine Quelle mit instabilen Kennungen kann dagegen unnötige Duplikate erzeugen.

Die Codec-Schnittstelle kapselt diese Besonderheiten. `rssv0` verarbeitet allgemeine JSON-Feeds und klassischen XML-RSS. `tazv0` lädt zusätzlich den vollständigen Artikel über den Feed-Link, wenn der Feed nur eine verkürzte Darstellung enthält. `screenv0` öffnet den ersten Link mit Chromium, wartet auf die Darstellung, entfernt bekannte Consent- und Cookie-Banner und speichert einen vollständigen PNG-Screenshot. Der Screenshot konserviert eine visuelle Momentaufnahme, nicht nur die extrahierten Textfelder.

Mit größerer Quellenflexibilität wachsen die dokumentarischen Unsicherheiten. Externe Seiten können dynamische Inhalte, personalisierte Elemente, nachgeladene Bilder oder wechselnde Werbeflächen enthalten. Ein Screenshot hängt von Browser, Viewport und Zeitpunkt ab. News-Hash behauptet daher nicht, eine Quelle vollständig oder dauerhaft abzubilden. Es archiviert die Fassung, die unter den gegebenen Abrufbedingungen beobachtet wurde.

6. Persistenz in JSONL und SQLite

JSONL und SQLite erfüllen komplementäre Funktionen. JSONL ist ein einfaches Archivformat: Jede Zeile enthält einen JSON-Datensatz, der mit Standardwerkzeugen gelesen, kopiert und versioniert werden kann. SQLite stellt Indizes, Abfragen und relationale Verknüpfungen für die WebUI bereit. Bilder werden für JSONL als Dateien unter `data/images/` referenziert und in SQLite zusätzlich als deduplizierte BLOB-Daten abgelegt.

Beide Speicher führen eine eigene Hash-Kette. Der letzte bekannte Hash, die bekannten Quellenkennungen und der Vorgängerwert werden pro Speicherformat ermittelt. Diese Entscheidung ist eine direkte Reaktion auf die unterschiedliche Dauerhaftigkeit der beiden Schreibwege. Ein Prozess kann zwischen einem Dateischreibvorgang und einer SQLite-Transaktion beendet werden. Anstatt eine nicht vorhandene globale Transaktion zu simulieren, macht die Anwendung die beiden Zustände unabhängig prüfbar.

Erreicht ein Shard eine Größe von 1 GB, wird eine neue nummerierte Datei begonnen. Die Dashboard-Vorschau liest aus Performancegründen nur den neuesten SQLite-Shard. Aggregierte Kennzahlen und Detailzugriffe können dagegen über alle Shards arbeiten. Für die Deduplizierung wird ebenfalls nur der neueste Shard je Speicherformat betrachtet. Diese Regeln sind ein Kompromiss zwischen Importgeschwindigkeit, Speicherwachstum und historischer Abfragefähigkeit.

Bei einer Schemaabweichung wird eine vorhandene SQLite-Tabelle nicht automatisch gelöscht. Sie wird nach einem Namen mit Zeitstempel umbenannt und durch eine Tabelle mit aktuellem Schema ersetzt. Dadurch bleibt der alte Zustand forensisch verfügbar. Eine vollständige Migration oder semantische Zusammenführung ist bewusst nicht Teil dieser Operation; sie würde die Gefahr erhöhen, historische Daten während des Programmstarts irreversibel zu verändern.

7. Formale Definition der Hash-Kette

Sei R_i ein normalisierter Datensatz und H_i sein gespeicherter Hash. Der erste Datensatz erhält als Vorgängerwert einen String aus 64 Nullen. Für jeden weiteren Datensatz wird der Hash des unmittelbar vorherigen Datensatzes als P_i übernommen. Das Hashmaterial besteht aus den fachlichen Feldern und diesem Vorgängerwert. Es wird als JSON-Objekt mit sortierten Schlüsseln, ohne zusätzliche Leerzeichen und in UTF-8 serialisiert.

$$\begin{aligned} P_0 &= 0^{64} \\ H_i &= \text{SHA-256}(\text{canonical_json}(R_i, P_i)) \\ P_{i+1} &= H_i \end{aligned}$$

Für den Standardcodec umfasst das Material unter anderem Autor, Inhalt, Codecname, Bilder, Vorgängerhash, Veröffentlichungszeit, Quellenkennung, Quell-URL und Titel. Der Abrufzeitpunkt wird gespeichert, fließt aber nicht in den Inhalts-Hash ein. Damit erzeugt ein späterer Abruf eines unveränderten veröffentlichten Inhalts nicht allein wegen des neuen Abrufzeitpunkts einen anderen Hash.

Die Validierung prüft zwei Beziehungen. Erstens muss der gespeicherte Vorgängerhash zum erwarteten Datensatz passen. Zweitens muss der gespeicherte Hash dem Ergebnis der erneuten Berechnung entsprechen. Eine Inhaltsänderung verletzt die zweite Beziehung; eine Umordnung oder das Einfügen an einer unerwarteten Stelle verletzt die Kettenbeziehung. Änderungen an einem frühen Datensatz wirken sich durch die Folge der Vorgängerwerte auf spätere Datensätze aus.

Das Modell hängt von einer stabilen kanonischen Repräsentation ab. Eine Änderung am Feldmapping, an der Serialisierung oder am Algorithmus kann daher nicht ohne Weiteres als normale Datenänderung behandelt werden. Für eine langfristige Weiterentwicklung wären explizite Schema- und Algorithmusversionen im Datensatz oder Manifest sinnvoll. Der gegenwärtige Entwurf dokumentiert die Feldzuordnung im Codec und deckt die Berechnung durch Tests ab.

8. Fehler, Abstürze und Wiederanlauf

Ein Archivdienst muss nicht nur den Erfolgsfall, sondern auch unvollständige und widersprüchliche Zustände beschreiben. Netzwerkfehler, ungültiges JSON, nicht erreichbare Artikel, fehlende Bilder und Browserprobleme werden pro Quelle behandelt. Die betroffene Quelle erhält einen Laufzeitfehler; andere Quellen laufen weiter. Die Fehlermeldung wird nicht Teil der Hash-Kette, weil sie ein flüchtiger Betriebszustand und kein archivierter Nachrichteninhalt ist.

Beim Prozessabbruch zwischen zwei Speicheroperationen kann JSONL bereits den neuen Datensatz enthalten, während SQLite noch auf dem vorherigen Kettenende steht. Die beiden Ketten sind dann nicht gleich lang, aber nicht deshalb automatisch ungültig. Ein Wiederanlauf muss die jeweils letzte gültige Kette fortsetzen. Die getrennte Verwaltung macht sichtbar, welcher Speicher voraus ist, und verhindert, dass eine beschädigte Seite die andere stillschweigend überschreibt.

Ein Neustart setzt die flüchtigen Fehlerzähler und das Laufzeitprotokoll zurück. Persistierte Datensätze, Bilder, Shards und Anchor-Dateien bleiben dagegen erhalten. Diese Unterscheidung ist für die Interpretation des Dashboards wichtig. Ein leerer Fehlerzähler bedeutet nicht, dass die Quelle historisch immer erfolgreich war; er bedeutet nur, dass der aktuelle Prozess seit seinem Start keine entsprechende Beobachtung gespeichert hat.

Die Fehlersemantik ist damit absichtlich konservativ. News-Hash versucht nicht, eine beschädigte Quelle automatisch zu reparieren oder historische Ketten nachträglich zu glätten. Eine Reparatur kann als neue technische Operation dokumentiert werden, aber sie darf nicht mit einer unveränderten ursprünglichen Archivspur verwechselt werden. Diese Haltung erhöht die Nachvollziehbarkeit auf Kosten einer scheinbar bequemerer automatischen Bereinigung.

9. Öffentliche Zeitverankerung

Nach einem erfolgreichen Quellenlauf kann für den UTC-Tag ein Manifest erzeugt werden. Es enthält die jeweils letzten Hashes der JSONL- und SQLite-Kette, jedoch keine Nachrichtentexte und keine Bilder. Das Manifest ist klein genug, um öffentlich veröffentlicht und unabhängig geprüft zu werden. Es repräsentiert den lokalen Archivzustand nicht vollständig, sondern über seine beiden Kettenendpunkte.

OpenTimestamps versieht die Manifestdatei mit einem attestierten Zeitpfad. Der Befehl `ots stamp` verarbeitet die Datei und erzeugt eine zugehörige `.ots`-Datei. Eine spätere Prüfung muss zunächst den lokalen Inhalt und die Kette verifizieren, dann die aktuellen Kettenwerte mit dem Manifest vergleichen und erst danach den Zeitnachweis prüfen. Nur diese Reihenfolge verhindert, dass ein gültiger Zeitstempel für einen Hash fälschlich als Beleg für einen beliebigen Inhalt interpretiert wird.

Die GitHub-Synchronisierung ergänzt die externe Zeitverankerung. Manifest und Proof-Datei werden in einem Repository versioniert veröffentlicht. GitHub liefert damit Auffindbarkeit und eine sichtbare Veröffentlichungshistorie; OpenTimestamps liefert den unabhängigen zeitlichen Bezug. Bestehende Dateien werden inhaltlich verglichen und unveränderte Dateien nicht erneut per API hochgeladen. So werden unnötige Git-Commits vermieden.

Die gewählte Konstruktion wahrt eine wichtige Datenminimierung. Der externe Dienst erhält keine Artikelinhalte. Dennoch ist ein Manifest nicht geheim: Hashwerte können unter bestimmten Umständen Informationen über bekannte Inhalte oder Zustände verraten. Die Entscheidung, welche Manifeste und welche lokalen Daten veröffentlicht werden, bleibt deshalb eine Frage des Betriebs- und Datenschutzkonzepts, nicht nur eine technische Voreinstellung.

10. WebUI und Beobachtbarkeit

Die WebUI stellt die technische Archivspur in einer für Menschen prüfbarer Form dar. Quellenkarten zeigen Anzahl der Meldungen, Bilder, Fehler und den Anchor-Status. Eine Meldungsliste unterstützt Quellfilter, Pagination und den Übergang zur Detailseite. Die Detailansicht verbindet Nachrichtentext, Zeitangaben, Hashes und gespeicherte Bilder. Damit werden nicht nur Metriken, sondern auch die konkreten Objekte sichtbar, auf die sich eine Integritätsprüfung bezieht.

Die Anwendung verarbeitet Zeitstempel intern in UTC. Erst im Browser werden sie in die lokale Zeitzone umgerechnet. Diese Trennung verhindert, dass der Rechner des Daemons und der Rechner einer lesenden Person dieselbe Meldung mit verschiedenen gespeicherten Zeitwerten versehen. Für wissenschaftliche Auswertungen ist dennoch zu dokumentieren, ob eine Anzeigzeit oder der kanonische UTC-Wert verwendet wurde.

Prometheus-Metriken ergänzen die fachliche Oberfläche um maschinenlesbare Betriebsdaten. Erfasst werden unter anderem konfigurierte Quellen, Datensätze, Bilder und Quellenfehler. Ein optionaler Heartbeat meldet den Prozessbetrieb an einen konfigurierten Dienst. Beide Mechanismen beantworten jedoch nur Betriebsfragen. Sie zeigen, ob der Dienst läuft oder welche Zähler er führt; sie prüfen nicht selbst, ob der Inhalt einer Kette korrekt ist.

Die WebUI verzichtet auf Benutzerverwaltung und Authentifizierung, weil der lokale Betrieb im Vordergrund steht. Diese Vereinfachung ist eine klare Einsatzgrenze. Für eine öffentliche Bereitstellung wären mindestens Netzwerksegmentierung, Reverse-Proxy-Schutz, Zugriffskontrolle und eine Prüfung der schreibenden Endpunkte erforderlich. Ein Dashboard ist kein Sicherheitsmechanismus, sondern eine Beobachtungsschnittstelle.

11. Verifikation und Reproduzierbarkeit

Eine unabhängige Verifikation kann in mehreren Stufen erfolgen. Zuerst wird geprüft, ob jede JSONL-Zeile syntaktisch lesbar ist und ob die SQLite-Tabellen dem erwarteten Schema entsprechen. Danach werden Datensätze in der gespeicherten Reihenfolge gelesen, die Vorgängerbeziehungen nachvollzogen und die Hashes aus dem dokumentierten Material neu berechnet. Die Ergebnisse der beiden Formate dürfen voneinander abweichen, wenn ein Speicher während eines Abbruchs voraus war; jedes Format muss aber seine eigene Kette erklären können.

Für die Prüfung eines Anchors werden anschließend Manifest und .ots-Datei benötigt. Der Prüfer vergleicht die im Manifest genannten Endhashes mit den lokal validierten Ketten. Erst wenn diese Werte übereinstimmen, besitzt der externe Zeitnachweis eine Aussage über die lokale Archivspur. Ein Proof ohne passende lokale Daten ist lediglich der Nachweis, dass ein bestimmter Hash existierte.

Reproduzierbarkeit umfasst nicht nur Softwareversionen. Feed-URL, Quellenkonfiguration, Codec, Abrufintervall, Netzwerkantwort, Browser-Version, Viewport und Zeitpunkt können die archivierte Fassung beeinflussen. JSON- und XML-Feeds lassen sich häufig strukturell reproduzieren; eine dynamische Zielseite oder ein Screenshot ist stärker kontingent. Die Architektur sollte diese Unterschiede dokumentieren, statt sie unter einer einheitlichen Genauigkeitsbehauptung zu verbergen.

Das Projekt unterstützt Reproduzierbarkeit durch ``pyproject.toml``, ``uv.lock``, Tests und Projektdokumentation. Für einen dauerhaften wissenschaftlichen Datensatz wären zusätzlich Konfigurationssnapshots, Prüfsummen der verwendeten Software und eine unveränderliche Kopie der Rohantworten empfehlenswert. Diese Erweiterungen sind mit zusätzlichen Speicher- und Datenschutzkosten verbunden und gehören daher in eine explizite Forschungs- oder Aufbewahrungsstrategie.

12. Evaluation durch Szenarien

Im Normalbetrieb wird ein neuer Feeddatensatz abgerufen, normalisiert, auf bekannte Quellenkennung geprüft, um Bilder oder Zusatzinhalte ergänzt, gehasht und in beide Speicher geschrieben. Die erwartete Kette wächst um genau ein Element pro Speicherformat. Die Dashboard-Liste zeigt den Datensatz; die Detailansicht kann Text und Bilder öffnen. Dieses Basisszenario prüft die Verbindung der Hauptkomponenten, sagt aber noch wenig über Fehlertoleranz aus.

Beim Wiederholungsabruf derselben Meldung erkennt die Deduplizierung die bekannte Quellenkennung und vermeidet einen weiteren Ketteneintrag. Wird eine neue Quellenfassung unter einer neuen Kennung geliefert, entsteht ein neuer Datensatz. Diese Regel ist einfach und effizient, verlagert aber einen Teil der Identitätssemantik in die Quelle. Eine Anwendung kann nicht allein aus einem Titel sicher ableiten, ob zwei redaktionelle Fassungen fachlich identisch sind.

Bei einem fehlerhaften Feed wird die Quelle als fehlerhaft markiert, während andere Quellen weiterlaufen. Bei einem Prozessabbruch zwischen JSONL- und SQLite-Schritt können die Ketten unterschiedliche Enden haben. Bei einer nachträglichen manuellen Änderung schlagen die Hashprüfungen fehl. Bei einer inkompatiblen Tabelle wird die alte Struktur umbenannt. Diese Szenarien zeigen verschiedene Schichten der Robustheit: Isolation, Erkennung, Erhaltung und kontrollierten Wiederanlauf.

Die Tests des Projekts decken Settings-Validierung, Feednormalisierung, Codecverhalten, Hash-Ketten, Sharding, SQLite-Schema, Anchoring, GitHub-Synchronisierung, Daemon-Polling und WebUI-Rendering ab. Screenshot-Tests laden Seiten mit `domcontentloaded` und einer kurzen Renderwartezeit statt mit `networkidle`. Damit wird die Testumgebung auch für Seiten mit dauerhaften Netzwerkverbindungen deterministischer.

13. Bedrohungsmodell und Datenschutz

Das Bedrohungsmodell umfasst unbemerkte lokale Änderungen, beschädigte Dateien, unvollständige Schreibvorgänge, fehlende Quellen und eine verfälschte Eingabe vor dem Hashing. Gegen nachträgliche lokale Änderungen bietet die Kette eine starke Sichtbarkeit, sofern der Prüfer auf eine vertrauenswürdige vorherige Kette oder ein veröffentlichtes Manifest zurückgreifen kann. Gegen einen Betreiber, der von Beginn an falsche Inhalte speichert, hilft die Kette nicht. Sie kann auch nicht beweisen, dass ein nicht abgerufener Artikel nie existiert hat.

Ein vollständiges Löschen des lokalen Archivs wird durch eine lokale Hash-Kette nicht verhindert. Dafür sind unabhängige Backups oder öffentlich veröffentlichte Nachweise erforderlich. Ein tägliches Manifest schützt außerdem nur den darin repräsentierten Kettenendpunkt. Die Granularität des Nachweises ist daher geringer als die Granularität einzelner Nachrichten. Diese Eigenschaft ist ein bewusster Kompromiss zwischen Datenschutz, Kosten und Prüfwert.

Nachrichten können personenbezogene Daten, sensible Aussagen oder urheberrechtlich geschützte Inhalte enthalten. Die Tatsache, dass ein Inhalt technisch gehasht werden kann, begründet kein Recht zu seiner dauerhaften Speicherung. Quellenwahl, Aufbewahrungsdauer, Zugriffsrechte und Löschverfahren

müssen rechtlich und redaktionell bewertet werden. OpenTimestamps reduziert zwar den externen Inhaltsabfluss, macht aber die lokale Verantwortung nicht überflüssig.

Auch die WebUI ist Teil der Bedrohungsfläche. Ohne Authentifizierung dürfen Abruf-, Medien- und Shutdown-Endpunkte nicht öffentlich erreichbar sein. Credentials für GitHub werden aus einer lokalen Datei gelesen und nicht geloggt; sichere Dateirechte und Secret-Verwaltung obliegen der Umgebung. Der Artikel versteht diese Punkte als Systemanforderungen und nicht als optionale Komfortmaßnahmen.

14. Grenzen der Aussagekraft

Die zentrale Aussage der Hash-Kette ist eng: Die gespeicherte Repräsentation entspricht der erwarteten Kette oder eine Abweichung lässt sich erkennen. Daraus folgt nicht, dass ein Artikel wahr, ausgewogen, vollständig oder frei von Fehlern ist. Ein lokales Archiv kann eine irreführende Quelle sehr zuverlässig unverändert bewahren. Integrität und Qualität sind unterschiedliche Eigenschaften und müssen in einer wissenschaftlichen Auswertung getrennt berichtet werden.

Auch die zeitliche Aussage ist begrenzt. Ein OpenTimestamps-Proof belegt die Existenz eines Hashes spätestens zu einem attestierten Zeitpunkt. Er beweist nicht, dass der darin referenzierte Nachrichteninhalt damals bereits öffentlich war, sofern keine weitere Quelle diese Verbindung dokumentiert. Der Proof ist ein Existenznachweis für eine Bytefolge, keine unabhängige redaktionelle Beglaubigung.

Die Codecarchitektur erhöht die Abdeckung, kann aber auch neue Unsicherheiten erzeugen. Der vollständige Abruf eines Artikels hängt von dessen Verfügbarkeit und der Interpretation durch den Codec ab. Ein Screenshot hängt von Browser, Bildschirmgröße, dynamischen Ressourcen und Consent-Zuständen ab. Deshalb muss die jeweils verwendete Methode zusammen mit dem Archivdatensatz ausgewertet werden. Ein Textrecord und ein visueller Snapshot sind verschiedene Beobachtungsobjekte.

Schließlich ist die Anwendung als lokaler Dienst keine globale Infrastruktur. Der Betrieb kann ausfallen, Quellen können sich verändern und eine einzelne Instanz kann nicht alle Perspektiven des Nachrichtenraums abbilden. Der Wert des Projekts liegt nicht in einer totalen Repräsentation, sondern in der transparenten Dokumentation eines begrenzten, technisch prüfbaren Ausschnitts.

15. Weiterentwicklung

Eine erste Weiterentwicklung wäre die explizite Versionierung von Schema, Hashalgorithmus und Codec. Damit könnten mehrere Kettengenerationen nebeneinander geprüft werden, ohne dass eine spätere Änderung der Kanonisierung als Datenkorruption erscheint. Ergänzend wäre ein eigenständiges Verifikationswerkzeug sinnvoll, das JSONL, SQLite, Manifest und Proof lesen kann, ohne den laufenden Daemon und seine Netzwerkabhängigkeiten zu starten.

Für die Langzeitarchivierung könnten unveränderliche oder geografisch getrennte Replikate eingesetzt werden. Ein Replikationsprotokoll müsste dabei nicht nur Dateien kopieren, sondern auch den Zeitpunkt und die Integritätsprüfung jeder Kopie dokumentieren. Die öffentliche GitHub-

Synchronisierung ist für kleine Manifeste geeignet, ersetzt aber keine belastbare Sicherungsstrategie für große Bild- und Nachrichtendaten.

Die Beobachtbarkeit könnte um Soll-Ist-Vergleiche für Abrufintervalle, fehlende Meldungen und ungewöhnliche Größenänderungen erweitert werden. Für Screenshots könnten standardisierte Viewports, Browser-Versionen, Ladezeiten und eine Liste geladener Ressourcen gespeichert werden. Dadurch würde die visuelle Archivierung besser vergleichbar, aber auch stärker an konkrete Softwareumgebungen gebunden.

Eine weitere Forschungsrichtung wäre die Untersuchung von Quellenvielfalt und zeitlicher Verzerrung. Das technische Archiv kann mehrere Quellen parallel sichtbar machen, bewertet deren journalistische Qualität aber nicht. Metadaten über Quelle, Codec und Abrufhäufigkeit könnten für eine spätere Medienanalyse genutzt werden, sofern sie nicht mit einer automatischen Objektivitätsbehauptung verwechselt werden.

16. Fazit

News-Hash zeigt einen möglichen Mittelweg zwischen flüchtiger Webbeobachtung und aufwendiger verteilter Archivierungsinfrastruktur. Ein lokaler Prozess ruft konfigurierte Quellen ab, normalisiert ihre Daten und legt sie in zwei praktisch unterschiedlichen Formaten ab. Hash-Ketten machen nachträgliche Änderungen an der lokalen Spur sichtbar. Ein tägliches Manifest und OpenTimestamps erweitern diese interne Prüfbarkeit um eine öffentliche zeitliche Referenz.

Die technische Konstruktion ist deshalb überzeugend, weil sie ihre Grenzen offenlegt. Sie verspricht keine Wahrheit, keine Vollständigkeit und keine automatische rechtliche Zulässigkeit. Sie verspricht eine präzise kleinere Aussage: Eine konkrete archivierte Fassung kann gegen ihre Kette und, über ein Manifest, gegen einen externen Zeitnachweis geprüft werden. Diese Bescheidenheit ist keine Schwäche, sondern Voraussetzung für eine belastbare Interpretation.

Als wissenschaftliches Artefakt verbindet das Projekt Datenmodell, Implementierung, Tests, Betrieb und Dokumentation. Die Schnittstellen zwischen Feed, Normalisierung, Codec, Hashing, Speicherung, Manifest und WebUI können einzeln untersucht und erweitert werden. Damit eignet sich News-Hash nicht nur als persönliches Archiv, sondern auch als nachvollziehbarer Ausgangspunkt für weitere Arbeiten zu digitaler Provenienz, Quellenkritik und langfristiger Nachrichtenüberlieferung.

Literatur und Projektdokumentation

[1] News-Hash-Projekt: *Architektur*, `project-docu/architecture.md`, Projektstand 2026.

[2] News-Hash-Projekt: *Anforderungen*, `project-docu/requirements.md`, Projektstand 2026.

[3] News-Hash-Projekt: *README und WebUI-Dokumentation*, Projektstand 2026.

[4] National Institute of Standards and Technology: *Secure Hash Standard (SHS)*, FIPS PUB 180-4.

[5] OpenTimestamps: *OpenTimestamps Documentation*, <https://opentimestamps.org/>.

Dieser Artikel beschreibt den Projektstand vom 17. August 2026. Die PDF wurde aus dieser HTML-Quelle mit Chromium im A4-Format erzeugt.